SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192421166
Name	DHILIBAN M

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

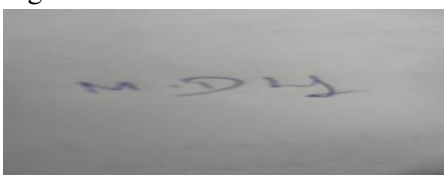
Question Count: 5

Code of Conduct

I DHILIBAN M/192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Q1. Job Fails Repeatedly: Uneven Input Splits, Idle Reducers

Identification of Error

The root issue is **poor input split / partition design**, not a hardware or cluster capacity problem. Splits are uneven in size (or key distribution is skewed), so a few map tasks run far longer than others, and the resulting map output keys are unevenly distributed across reducers — some reducers get almost no data and sit idle while one or two reducers ("stragglers") do most of the work.

Cause Analysis

- **Skewed input data:** Source files vary drastically in size, or mapreduce.input.fileinputformat.split.minsize/maxsize is left at defaults, letting HDFS block boundaries (not logical record boundaries) define splits.
- **Skewed key distribution:** The default HashPartitioner sends output to reducers based on key hash — if one key (e.g., one customer ID or one log source) dominates the dataset, that reducer becomes overloaded while others stay idle.
- **No combiner:** Without a combiner, all map output — including from small/skewed splits — travels to reducers, amplifying shuffle imbalance.

Corrective Action

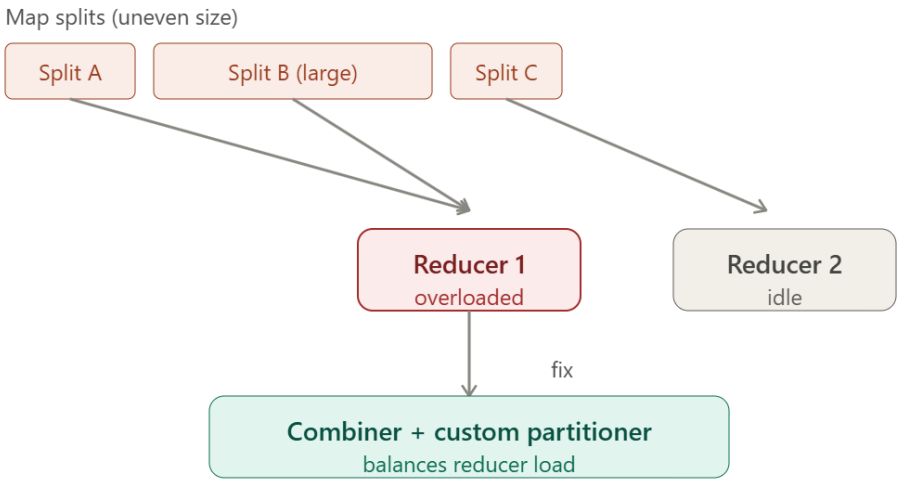
- **Tune split sizing:** Set mapreduce.input.fileinputformat.split.maxsize explicitly so splits are close to uniform, independent of raw file size.
- **Use a Combiner** to pre-aggregate map output and shrink shuffle volume before it reaches reducers.
- **Custom Partitioner:** Replace default hash partitioning with a partitioner aware of key skew (e.g., salting hot keys, or range-partitioning) so no single reducer is overloaded.
- **Enable speculative execution** (mapreduce.reduce.speculative=true) so a straggling reducer's work is duplicated elsewhere and idle capacity is used productively.
- **Pre-analyze key distribution** (sampling) before the job to choose reducer count and partitioning strategy proactively.

Hadoop Concepts Applied

InputFormat/split sizing, Combiner, Partitioner, speculative execution — all core MapReduce tuning levers.

Clarity Summary

Uneven splits + skewed partitioning → some reducers overloaded, others idle → fix at the split and partition layer, not by adding more nodes.



Q2. Frequent Data Unavailability After a Single Node Failure

Identification of Error

This is a **replication/NameNode design error**: either the replication factor is too low (RF=1 or effectively 1 due to failed re-replication), or — more likely given "single node failure" causes an outage — the **NameNode itself is not deployed in HA**, meaning a single NameNode failure takes down metadata access for the entire filesystem, even though DataNodes are fine.

Cause Analysis

- **No NameNode HA:** A single NameNode is a textbook single point of failure — if it goes down, HDFS is unavailable cluster-wide regardless of DataNode replication.
- **Replication factor too low or rack-unaware placement:** If RF=1–2, or replicas are all placed on the same rack/node group, a single node/rack failure can make blocks genuinely unavailable, not just under-replicated.
- **Balancer/re-replication not running:** Even at RF=3, if under-replicated blocks aren't proactively re-replicated after a failure, repeated failures over time erode redundancy.

Corrective Action

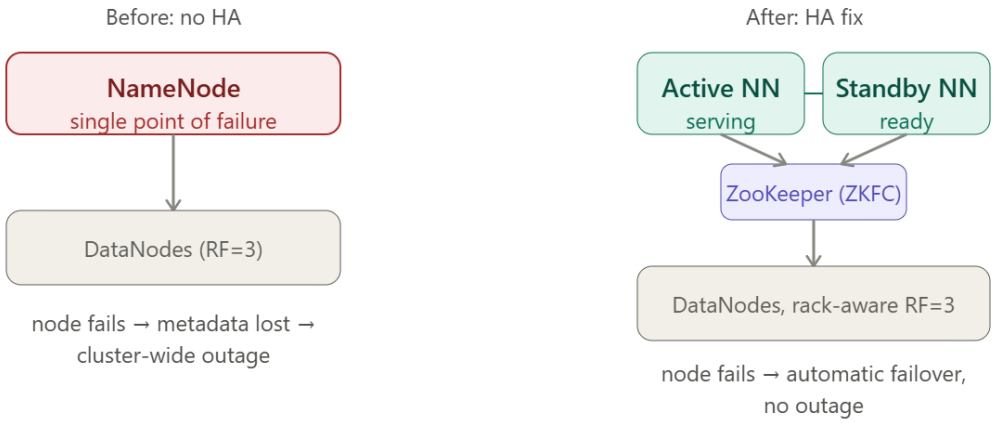
- **Deploy NameNode HA**: Active/Standby NameNode pair with Quorum Journal Manager (QJM) and ZooKeeper-based automatic failover (ZKFC) — this alone fixes "one node failure = full outage."
- **Set RF=3 with rack-aware placement** so no single node or rack holds all copies of a block.
- **Enable the HDFS Balancer and default re-replication** so the NameNode automatically restores RF after any node loss.
- **Monitor under-replicated block count** as a standing operational metric, not just after incidents.

Hadoop Concepts Applied

NameNode HA (QJM, ZKFC), rack-awareness, replication factor, HDFS balancer/re-replication.

Clarity Summary

If DataNode failure alone should be tolerable, the true fault is architectural: no HA on the metadata layer, and/or replication placement that doesn't survive a single failure domain.



Q3. NiFi/ETL Ingestion Creates Duplicate Records in the Analytics Store

Identification of Error

The pipeline lacks **idempotency and exactly-once (or at-least-once + dedup) guarantees**. NiFi/ETL retries, restarts, or overlapping scheduled runs are re-delivering the same records, and the analytics store has no mechanism to recognize and reject repeats.

Cause Analysis

- **At-least-once delivery without deduplication:** NiFi flow files are retried automatically on failure (network blip, downstream timeout) — this is correct NiFi behavior, but if the write to the sink isn't idempotent, retries create duplicates.
- **No unique key / natural key on the sink table:** Without a primary key or unique constraint, the store happily inserts the same record twice.
- **Overlapping scheduled ingestion windows:** If a NiFi GetFile/DB-pull processor's schedule overlaps with a slow-running previous execution (e.g., a Sqoop incremental job that hasn't finished before the next trigger), the same source rows get pulled twice.
- **CDC replaying already-processed events:** If change-data-capture offsets/checkpoints aren't committed correctly, a restart can replay already-ingested changes.

Corrective Action

- **Enforce a unique/primary key** on the target table (e.g., a composite of source_id + timestamp) so duplicate inserts are rejected or upserted (MERGE/UPSERT instead of blind INSERT).
- **Make the pipeline idempotent:** design ETL/NiFi flows so re-running the same batch produces the same end state (upsert semantics) rather than appending.
- **Use NiFi's built-in provenance and DetectDuplicate processor** (backed by a distributed cache) to filter already-seen flow files before they reach the sink.

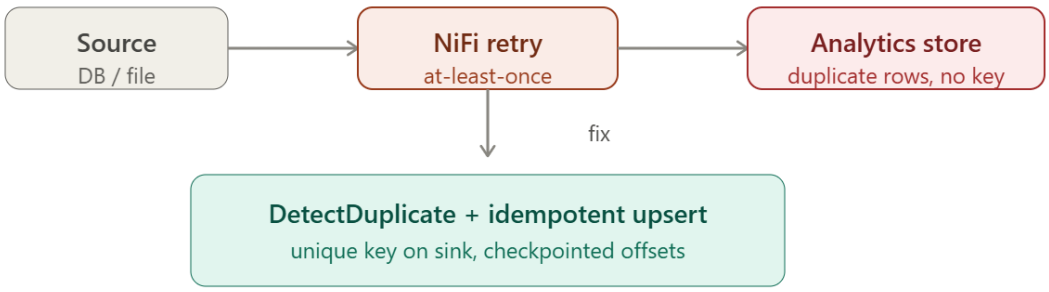
- **Commit CDC offsets transactionally** with the write, so a restart resumes exactly where it left off rather than reprocessing.
- **Prevent overlapping runs** by adding a scheduling lock/mutex (e.g., NiFi's FlowFileConcurrency controls, or an external check that the previous run has completed before triggering the next).

Hadoop/Integration Concepts Applied

NiFi flow file lifecycle, provenance, DetectDuplicate, idempotent upserts, CDC offset checkpointing.

Clarity Summary

Duplication is a delivery-semantics problem: at-least-once delivery is fine as long as the write path is idempotent — the fix is deduplication logic and unique constraints, not "make ingestion more reliable."



Q4. YARN Resource Starvation Under Concurrent Multi-User Job Submission

Identification of Error

The cluster is very likely running the **default FIFO Scheduler**, or a Capacity/Fair Scheduler configured **without preemption and without per-queue minimum guarantees** — so the first large job(s) submitted consume all containers, and subsequent users' jobs queue indefinitely (starvation) even though the cluster has ongoing capacity turnover.

Cause Analysis

- **FIFO Scheduler behavior:** Jobs are served strictly in submission order; a single large job can occupy the whole cluster until completion, starving everyone behind it.
- **No minimum resource guarantees per user/queue:** Even under Capacity/Fair Scheduler, if queues aren't configured with minimum shares, a burst of jobs from one user can dominate.
- **No preemption enabled:** Without preemption, once a queue is over its fair share, YARN won't reclaim containers from it even if another queue is starved.
- **Container size mismatch:** If jobs request oversized containers relative to available node capacity, fewer jobs can run concurrently, worsening perceived starvation.

Corrective Action

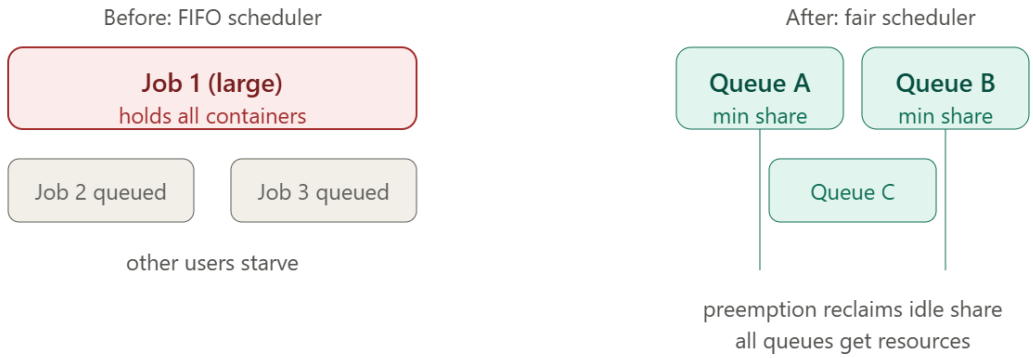
- **Switch to Fair Scheduler (or Capacity Scheduler with well-tuned queues)** with per-user or per-team queues, each given a **minimum resource share**.
- **Enable preemption** (yarn.scheduler.fair.preemption=true) with a bounded fairSharePreemptionTimeout, so a queue that's been starved below its minimum share for too long automatically reclaims containers.
- **Set sensible queue weights** reflecting business priority (e.g., production ETL > ad hoc analytics) while still guaranteeing every queue a floor.
- **Right-size container requests** (yarn.scheduler.minimum-allocation-mb/vcores) to avoid large, hard-to-schedule containers blocking smaller jobs.
- **Set per-queue/per-user max resource caps** so no single submission burst can exceed a fair ceiling.

Hadoop Concepts Applied

YARN scheduler types (FIFO vs Fair vs Capacity), queue configuration, preemption, container sizing.

Clarity Summary

Starvation under concurrent submission is a scheduler-policy gap, not a capacity shortage — moving from FIFO (or an unpreemptive queue setup) to a weighted, preemptive Fair/Capacity Scheduler resolves it directly.



Q5. Backup Restores Outdated Data After Recovery

Identification of Error

The design flaw is in the **backup/snapshot timing and consistency model** — the backup mechanism is either taking **infrequent point-in-time snapshots** with a large recovery point objective (RPO) gap, or relying on **asynchronous replication** whose lag isn't accounted for, so the "backup" reflects an earlier state than the last acknowledged writes.

Cause Analysis

- **Infrequent snapshot schedule:** If HDFS snapshots or distcp-based backups run, say, once every 24 hours, any restore after a failure loses up to a day of writes — this looks like "outdated data" but is really an RPO/scheduling gap.

- **Asynchronous replication lag:** If backups are streamed to a secondary cluster/cold storage asynchronously, a failure at the primary can occur before the latest writes have propagated, so the backup is behind by the lag window.
- **No write-ahead/journal-based consistency:** Without journaling (e.g., NameNode edit logs replayed correctly) tied into the backup process, the backup can capture a filesystem state that doesn't correspond to the most recent committed writes.
- **Backup validated only for completion, not recency:** If monitoring only checks "did the backup job succeed" and not "how far behind is this backup," staleness goes undetected until a real recovery is needed.

Corrective Action

- **Shrink the backup interval / move to continuous or near-real-time replication** (e.g., HDFS snapshot diffs pushed more frequently, or CDC-based continuous replication to the DR target) to reduce RPO.
- **Tie backups to the NameNode edit log / journal**, ensuring the backup always reflects the last durably committed transaction, not an arbitrary snapshot point.
- **Track and alert on replication lag** as a first-class metric (e.g., seconds/minutes behind primary), not just job success/failure.
- **Use HDFS Snapshots for point-in-time consistency** combined with incremental distcp (diff-based) to keep the secondary copy current with minimal overhead.
- **Test recovery drills regularly** (restore-and-verify) to catch staleness before it matters in a real incident, rather than discovering it during an actual failure.

Hadoop Concepts Applied

HDFS Snapshots, edit log/journal consistency, incremental distcp, RPO/lag monitoring.

Clarity Summary

"Restores outdated data" almost always traces to an RPO gap — backups too infrequent or too asynchronous relative to the write rate — the fix is tightening backup frequency/consistency and actively monitoring lag, not simply "having a backup."